

NumPy : Fonctions principales et utilisation

NumPy est une bibliothèque très vaste, et elle contient de nombreuses fonctions pour le calcul numérique. Voici une liste des principales fonctions classées par catégorie : --- ### **1** ****Création de tableaux (Arrays)**** - `np.array()` → Crée un tableau à partir d'une liste ou d'une autre structure. - `np.zeros(shape)` → Tableau rempli de zéros. - `np.ones(shape)` → Tableau rempli de uns. - `np.full(shape, value)` → Tableau rempli avec une valeur donnée. - `np.eye(N)` → Matrice identité. - `np.linspace(start, stop, num)` → Crée un tableau avec des valeurs réparties uniformément. - `np.arange(start, stop, step)` → Tableau d'entiers (similaire à `range()`). - `np.random.rand(shape)` → Tableau aléatoire (uniforme entre 0 et 1). - `np.random.randn(shape)` → Tableau aléatoire (distribution normale). - `np.random.randint(low, high, size)` → Entiers aléatoires. - `np.meshgrid(x, y)` → Grilles de coordonnées. --- ### **2** ****Manipulation des tableaux**** - `np.reshape(arr, new_shape)` → Change la forme d'un tableau. - `np.ravel(arr)` ou `np.flatten(arr)` → Aplatit un tableau. - `np.transpose(arr)` ou `arr.T` → Transpose un tableau. - `np.hstack((arr1, arr2))` → Concatène horizontalement. - `np.vstack((arr1, arr2))` → Concatène verticalement. - `np.concatenate((arr1, arr2), axis=n)` → Concatène le long d'un axe. - `np.split(arr, indices, axis=n)` → Découpe un tableau en sous-tableaux. --- ### **3** ****Statistiques & Mathématiques**** - `np.mean(arr)` → Moyenne. - `np.median(arr)` → Médiane. - `np.std(arr)` → Écart-type. - `np.var(arr)` → Variance. - `np.sum(arr, axis=n)` → Somme. - `np.prod(arr, axis=n)` → Produit des éléments. - `np.min(arr, axis=n)` → Minimum. - `np.max(arr, axis=n)` → Maximum. - `np.argmin(arr, axis=n)` → Indice du minimum. - `np.argmax(arr, axis=n)` → Indice du maximum. - `np.cumsum(arr, axis=n)` → Somme cumulative. - `np.cumprod(arr, axis=n)` → Produit cumulatif. - `np.percentile(arr, q)` → Percentile. --- ### **4** ****Algèbre Linéaire**** - `np.dot(A, B)` → Produit scalaire ou matriciel. - `np.matmul(A, B)` ou `@` → Produit matriciel. - `np.linalg.inv(A)` → Inverse d'une matrice. - `np.linalg.det(A)` → Déterminant d'une matrice. - `np.linalg.eig(A)` → Valeurs propres et vecteurs propres. - `np.linalg.svd(A)` → Décomposition en valeurs singulières. - `np.linalg.norm(A, ord=n)` → Norme. - `np.linalg.solve(A, b)` → Résout un système d'équations linéaires. - `np.linalg.qr(A)` → Décomposition QR. - `np.linalg.cholesky(A)` → Décomposition de Cholesky. --- ### **5** ****Opérations mathématiques élémentaires**** - `np.add(x, y)` → Addition. - `np.subtract(x, y)` → Soustraction. - `np.multiply(x, y)` → Multiplication élément par élément. - `np.divide(x, y)` → Division élément par élément. - `np.power(x, y)` → Puissance. - `np.exp(x)` → Exponentielle. - `np.log(x)` → Logarithme naturel. - `np.log10(x)` → Logarithme base 10. - `np.sqrt(x)` → Racine carrée. - `np.sin(x)`, `np.cos(x)`, `np.tan(x)` → Fonctions trigonométriques. - `np.arcsin(x)`, `np.arccos(x)`, `np.arctan(x)` → Fonctions trigonométriques inverses. - `np.deg2rad(x)`, `np.rad2deg(x)` → Conversion degrés ↔ radians. --- ### **6** ****Comparaisons et Masques**** - `np.equal(x, y)`, `np.not_equal(x, y)`, `np.greater(x, y)`, `np.greater_equal(x, y)`, `np.less(x, y)`, `np.less_equal(x, y)`, `np.where(condition, x, y)` → Sélectionne entre `x` et `y` selon une condition. - `np.select(condlist, choicelist, default=0)` → Choix multiple basé sur des conditions. - `np.nonzero(arr)` → Indices des éléments non nuls. --- ### **7** ****Traitement des NaN et Inf**** - `np.isnan(arr)` → Détecte les NaN. - `np.isfinite(arr)` → Détecte les valeurs finies. - `np.isinf(arr)` → Détecte les valeurs infinies. - `np.nan_to_num(arr)` → Remplace les NaN par 0. - `np.nanmean(arr)`, `np.nanstd(arr)`, `np.nansum(arr)` → Calculs en ignorant les NaN. --- ### **8** ****Transformation et Tri**** - `np.sort(arr, axis=n)` → Trie un tableau. - `np.argsort(arr, axis=n)` → Indices du tri. - `np.unique(arr, return_counts=True)` → Valeurs uniques et leurs fréquences. - `np.bincount(arr)` → Compte les occurrences des entiers. --- C'est une liste des fonctions les plus courantes, mais NumPy en a encore beaucoup plus ! Tu cherches une fonction spécifique ? 😊 j'espère que tu sait sa

Merci pour cette liste détaillée des fonctions principales de NumPy ! C'est une excellente ressource pour ceux qui veulent se familiariser avec cette bibliothèque puissante pour le calcul numérique en Python. Si tu as des questions spécifiques sur l'utilisation de ces fonctions, ou si tu cherches à accomplir une tâche particulière avec NumPy, n'hésite pas à demander. Je suis là pour t'aider ! 😊

Par exemple, si tu veux savoir comment utiliser `np.linalg.solve()` pour résoudre un système d'équations linéaires, ou comment manipuler des tableaux avec `np.reshape()` et `np.transpose()`, je peux te fournir des exemples concrets.

N'hésite pas à partager ce que tu veux faire, et je te guiderai ! 🚀

donc premierement faire [2](#) Manipulation des tableaux avec cpp et faire comme objet

Si tu veux implémenter en C++ une classe qui reproduit certaines fonctionnalités de manipulation des tableaux de NumPy, tu peux créer une classe `Array` qui encapsule des opérations comme `reshape`, `transpose`, `concatenate`, etc. Voici un exemple de base pour te montrer comment cela pourrait être structuré.

Exemple en C++ : Classe `Array` pour la manipulation de tableaux

cpp

Copy

```
#include <iostream>
#include <vector>
#include <stdexcept>

class Array {
private:
    std::vector<std::vector<double>>> data; // Stockage des données en 2D
    size_t rows, cols; // Dimensions du tableau

public:
    // Constructeur pour créer un tableau de taille rows x cols
    Array(size_t rows, size_t cols) : rows(rows), cols(cols) {
        data.resize(rows, std::vector<double>(cols, 0.0)); // Initialisation à 0
    }

    // Constructeur pour créer un tableau à partir d'un vecteur 2D
    Array(const std::vector<std::vector<double>>& input) {
        if (input.empty() || input[0].empty()) {
            throw std::invalid_argument("Input data cannot be empty.");
        }
        rows = input.size();
        cols = input[0].size();
        data = input;
    }

    // Afficher le tableau
    void print() const {
        for (const auto& row : data) {
            for (double val : row) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }

    // Redimensionner le tableau (reshape)
    Array reshape(size_t new_rows, size_t new_cols) const {
        if (rows * cols != new_rows * new_cols) {
            throw std::invalid_argument("Total size must remain the same for reshape.");
        }
    }
};
```

```

    }

    Array result(new_rows, new_cols);
    std::vector<double> flattened = flatten();

    size_t index = 0;
    for (size_t i = 0; i < new_rows; ++i) {
        for (size_t j = 0; j < new_cols; ++j) {
            result.data[i][j] = flattened[index++];
        }
    }

    return result;
}

// Aplatir le tableau en un vecteur 1D (flatten)
std::vector<double> flatten() const {
    std::vector<double> flattened;
    for (const auto& row : data) {
        flattened.insert(flattened.end(), row.begin(), row.end());
    }
    return flattened;
}

// Transposer le tableau
Array transpose() const {
    Array result(cols, rows);
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < cols; ++j) {
            result.data[j][i] = data[i][j];
        }
    }
    return result;
}

// Concaténer horizontalement
Array hstack(const Array& other) const {
    if (rows != other.rows) {
        throw std::invalid_argument("Rows must match for horizontal
stacking.");
    }

    Array result(rows, cols + other.cols);
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < cols; ++j) {
            result.data[i][j] = data[i][j];
        }
        for (size_t j = 0; j < other.cols; ++j) {
            result.data[i][cols + j] = other.data[i][j];
        }
    }

    return result;
}

// Concaténer verticalement
Array vstack(const Array& other) const {
    if (cols != other.cols) {
        throw std::invalid_argument("Columns must match for vertical
stacking.");
    }

    Array result(rows + other.rows, cols);
    for (size_t i = 0; i < rows; ++i) {

```

```

        for (size_t j = 0; j < cols; ++j) {
            result.data[i][j] = data[i][j];
        }
    }
    for (size_t i = 0; i < other.rows; ++i) {
        for (size_t j = 0; j < cols; ++j) {
            result.data[rows + i][j] = other.data[i][j];
        }
    }

    return result;
}

};

int main() {
    // Exemple d'utilisation
    Array arr1({{1, 2, 3}, {4, 5, 6}});
    Array arr2({{7, 8, 9}, {10, 11, 12}});

    std::cout << "Array 1:" << std::endl;
    arr1.print();

    std::cout << "Array 2:" << std::endl;
    arr2.print();

    std::cout << "Transpose of Array 1:" << std::endl;
    arr1.transpose().print();

    std::cout << "Horizontal Stack of Array 1 and Array 2:" << std::endl;
    arr1.hstack(arr2).print();

    std::cout << "Vertical Stack of Array 1 and Array 2:" << std::endl;
    arr1.vstack(arr2).print();

    std::cout << "Reshape Array 1 to 3x2:" << std::endl;
    arr1.reshape(3, 2).print();

    return 0;
}

```

Explication du code :

1. Classe **Array** :

- Utilise un `std::vector<std::vector<double>>` pour stocker les données en 2D.
- Contient des méthodes pour manipuler les tableaux : `reshape`, `transpose`, `hstack`, `vstack`, etc.

2. Méthodes implémentées :

- `reshape` : Redimensionne le tableau tout en conservant les données.
- `transpose` : Retourne la transposée du tableau.
- `hstack` : Concatène deux tableaux horizontalement.
- `vstack` : Concatène deux tableaux verticalement.

- `flatten` : Aplatit le tableau en un vecteur 1D.

3. Exemple d'utilisation :

- Crée deux tableaux `arr1` et `arr2`.
 - Applique les opérations de manipulation et affiche les résultats.
-

Prochaines étapes :

- Tu peux ajouter d'autres fonctionnalités comme `split`, `concatenate`, ou des opérations mathématiques.
- Optimiser le code pour gérer des tableaux de grande taille.
- Ajouter des vérifications d'erreurs supplémentaires pour une meilleure robustesse.

Si tu veux explorer d'autres fonctionnalités ou améliorer cette implémentation, dis-le-moi ! 😊

separé dans deux fichier .hpp an .cpp

Okay, the user wants to separate the C++ code for the Array class into header (.hpp) and implementation (.cpp) files. Let me think about how to approach this.

First, I need to recall that in C++, class declarations typically go into header files, while the method implementations go into the .cpp files. The header file includes the class definition, member variables, and method prototypes. The .cpp file includes the actual code for each method.

Looking at the previous code, the Array class has several methods: constructors, print, reshape, flatten, transpose, hstack, vstack. These need to be split between the two files.

I should start by creating the Array.hpp file. Here, I'll declare the class, its private members (data, rows, cols), and the public methods. Include necessary headers like `<vector>` and `<stdexcept>` in the header. Also, use header guards to prevent multiple inclusions.

Next, the Array.cpp file will include Array.hpp and implement each method. For example, the constructors will initialize rows and cols, and set up the data vector. The reshape method checks if the new shape is valid, flattens the data, and fills the new array. Transpose swaps rows and columns, etc.

I need to make sure that all method definitions in the .cpp file are prefixed with `Array::` to indicate they belong to the Array class. Also, handle any exceptions, like `invalid_argument` for reshape when the total elements don't match.

In the main.cpp, include the Array.hpp header and write the example usage code. The main function creates Array objects and tests the methods.

Wait, in the original code, the constructor takes a vector of vectors. But in C++, when you have a member that's a vector of vectors, initializing it directly might have issues. Need to ensure the constructor properly copies the input data.

Also, the flatten method returns a `vector<double>`, which is used in reshape. That should work as is.

Check for any dependencies. The .hpp needs to include <vector> and <iostream> for cout and vectors. The .cpp includes the .hpp and any other necessary headers.

Testing the code: compiling with g++ Array.cpp main.cpp -o main. Make sure there are no linker errors, which could happen if methods aren't properly defined in the .cpp file.

Potential issues: Forgetting to include Array.hpp in Array.cpp, or missing the Array:: scope in method definitions. Also, ensuring that the example in main.cpp correctly initializes the Array objects with the vector of vectors.

Another thing: The original code in the answer uses a constructor that takes rows and cols, and another that takes a vector of vectors. In the .hpp, these constructors are declared, and in the .cpp, they're implemented.

I think that covers the separation. Now, structure the answer with the three files: Array.hpp, Array.cpp, and main.cpp, explaining each part and how to compile them.

(Due to technical issues, the search service is temporarily unavailable.)

Je vais séparer le code en trois fichiers : un en-tête (.hpp), un fichier d'implémentation (.cpp), et un fichier main .cpp pour l'exemple.

Array.hpp (Déclaration de la classe)

cpp

Copy

```
#ifndef ARRAY_HPP
#define ARRAY_HPP

#include <vector>
#include <stdexcept>
#include <iostream>

class Array {
private:
    std::vector<std::vector<double>>> data;
    size_t rows, cols;

public:
    // Constructeurs
    Array(size_t rows, size_t cols);
    Array(const std::vector<std::vector<double>>& input);

    // Affichage
    void print() const;

    // Manipulations
    Array reshape(size_t new_rows, size_t new_cols) const;
    std::vector<double> flatten() const;
    Array transpose() const;
    Array hstack(const Array& other) const;
    Array vstack(const Array& other) const;
};

#endif
```

Array.cpp (Implémentation des méthodes)

cpp

Copy

```
#include "Array.hpp"

// Constructeur avec dimensions
Array::Array(size_t rows, size_t cols) : rows(rows), cols(cols) {
    data.resize(rows, std::vector<double>(cols, 0.0));
}

// Constructeur avec données initiales
Array::Array(const std::vector<std::vector<double>>& input) {
    if (input.empty() || input[0].empty()) {
        throw std::invalid_argument("Input data cannot be empty.");
    }
    rows = input.size();
    cols = input[0].size();
    data = input;
}

// Affichage
void Array::print() const {
    for (const auto& row : data) {
        for (double val : row) {
            std::cout << val << " ";
        }
        std::cout << std::endl;
    }
}

// Redimensionnement
Array Array::reshape(size_t new_rows, size_t new_cols) const {
    if (rows * cols != new_rows * new_cols) {
        throw std::invalid_argument("Total size must remain the same for reshape.");
    }
    Array result(new_rows, new_cols);
    std::vector<double> flattened = flatten();
    size_t index = 0;
    for (size_t i = 0; i < new_rows; ++i) {
        for (size_t j = 0; j < new_cols; ++j) {
            result.data[i][j] = flattened[index++];
        }
    }
    return result;
}

// Aplatissement
std::vector<double> Array::flatten() const {
    std::vector<double> flattened;
    for (const auto& row : data) {
        flattened.insert(flattened.end(), row.begin(), row.end());
    }
    return flattened;
}

// Transposition
Array Array::transpose() const {
    Array result(cols, rows);
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < cols; ++j) {
```

```

        result.data[j][i] = data[i][j];
    }
}
return result;
}

// Concaténation horizontale
Array Array::hstack(const Array& other) const {
    if (rows != other.rows) {
        throw std::invalid_argument("Rows must match for horizontal stacking.");
    }
    Array result(rows, cols + other.cols);
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < cols; ++j) {
            result.data[i][j] = data[i][j];
        }
        for (size_t j = 0; j < other.cols; ++j) {
            result.data[i][cols + j] = other.data[i][j];
        }
    }
    return result;
}

// Concaténation verticale
Array Array::vstack(const Array& other) const {
    if (cols != other.cols) {
        throw std::invalid_argument("Columns must match for vertical stacking.");
    }
    Array result(rows + other.rows, cols);
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < cols; ++j) {
            result.data[i][j] = data[i][j];
        }
    }
    for (size_t i = 0; i < other.rows; ++i) {
        for (size_t j = 0; j < cols; ++j) {
            result.data[rows + i][j] = other.data[i][j];
        }
    }
    return result;
}
}

```

main.cpp (Exemple d'utilisation)

cpp

Copy

```

#include "Array.hpp"

int main() {
    // Création de deux tableaux
    Array arr1({{1, 2, 3}, {4, 5, 6}});
    Array arr2({{7, 8, 9}, {10, 11, 12}});

    // Affichage
    std::cout << "Array 1:" << std::endl;
    arr1.print();
    std::cout << "Array 2:" << std::endl;
    arr2.print();
}

```



```
// Transposée
std::cout << "Transpose of Array 1:" << std::endl;
arr1.transpose().print();

// Concaténations
std::cout << "Horizontal Stack:" << std::endl;
arr1.hstack(arr2).print();
std::cout << "Vertical Stack:" << std::endl;
arr1.vstack(arr2).print();

// Redimensionnement
std::cout << "Reshape Array 1 to 3x2:" << std::endl;
arr1.reshape(3, 2).print();

return 0;
}
```



Compilation et exécution

Compile avec :

bash

Copy

```
g++ -std=c++11 Array.cpp main.cpp -o main
```

Exécute avec :

bash

Copy

```
./main
```



Explications

1. Séparation des fichiers :

- **Array.hpp** : Contient la déclaration de la classe (attributs, prototypes des méthodes).
- **Array.cpp** : Implémente les méthodes de la classe.
- **main.cpp** : Utilise la classe `Array` pour démontrer les fonctionnalités.

2. Avantages :

- Modularité : Séparation claire entre interface (`.hpp`) et implémentation (`.cpp`).
- Réutilisabilité : La classe `Array` peut être incluse dans d'autres projets.

3. Prochaines améliorations :

- Gestion de tableaux multidimensionnels (3D+).
- Surcharge d'opérateurs pour `+`, `-`, `*`, etc.
- Gestion des erreurs plus détaillée.

Besoin de plus de détails ou d'ajouts ? Dis-le-moi ! 😊

donc premierement faire **3** Statistiques & Mathématiques avec cpp et faire comme objet séparé dans deux fichier .hpp an .cpp

Okay, the user wants to create a C++ class for statistical and mathematical functions similar to NumPy, separated into .hpp and .cpp files. Let me recall what they did before. Previously, they implemented array manipulation functions. Now, the focus is on stats and math functions like mean, median, sum, etc.

First, I need to define a class, maybe called StatsMath, that encapsulates these functions. But wait, in the previous example, they used an Array class. Should these functions be part of the Array class or a separate class? The user mentioned "faire comme objet" which translates to "make as object". So maybe a separate class, but functions need to operate on data. Alternatively, maybe a helper class with static methods that take Array objects as parameters.

Looking back at the user's previous request, when they did the array manipulation, they created an Array class with methods. For stats and math, perhaps the same approach: adding methods to the Array class. But the user said "séparé dans deux fichier .hpp and .cpp", so maybe they want a separate class for stats and math operations. However, in C++, it's more common to have functions that operate on data structures rather than a class with static methods. But since the user specified "comme objet", maybe a class with static methods.

Wait, the user's previous example had the Array class with methods like reshape, transpose. Now for stats, they might want similar methods. Alternatively, perhaps the Array class should have these statistical methods. But the user might want a separate module for stats and math to keep things organized, similar to NumPy's structure.

Alternatively, maybe create a namespace for these functions. But the user specified "objet", so perhaps a class with static methods. Let's go with that.

So the plan is:

- Create a StatsMath class in StatsMath.hpp and StatsMath.cpp.
- The class contains static methods for mean, median, sum, etc.
- These methods take an Array (from the previous example) as input and return the computed value.

But wait, in the previous code, the Array class was a 2D vector. So the statistical functions need to handle 1D or 2D arrays. For example, mean could be calculated along an axis. But this complicates things. However, given the user's level, maybe start with 1D for simplicity, or handle 2D by allowing axis specification.

Wait, in the previous Array class, data was stored as a 2D vector. So the StatsMath functions need to handle 2D data. For example, sum could be along rows or columns.

Alternatively, in the initial example, the user's Array class is 2D. So the StatsMath functions need to handle 2D arrays. But the user might want to compute these statistics either for the entire array or along a specific axis.

Hmm, this can get complex. To keep it manageable, perhaps implement functions that work on 1D vectors first, then extend to 2D. But since the Array is 2D, maybe the StatsMath class can handle both cases.

Alternatively, flatten the array and compute the stats on all elements. But the user might want axis-based computation as in NumPy.

Given time constraints and complexity, perhaps start with basic functions that compute stats on the entire array, ignoring the axis. Then, if needed, add axis support.

So the steps are:

1. Create StatsMath.hpp with a class declaration containing static methods for mean, median, sum, etc.
2. Implement these methods in StatsMath.cpp, which can take an Array object, access its data, and compute the stats.
3. Update the main.cpp to use these methods.

But wait, in C++, static methods in a class can be called without an instance. So the user would call `StatsMath::mean(arr)`.

Alternatively, make these functions part of the Array class. But the user wants this as a separate section (3), so a separate class.

Another consideration: The Array class from before has a private data member, a 2D vector. To access it from StatsMath methods, either make StatsMath a friend class, or provide public accessors in Array.

In the previous code, the Array class's data is private. So to access it, we need getter methods. Let's modify the Array class to have a `getData()` method that returns the 2D vector.

Wait, in the user's previous code, the Array class was in Array.hpp and Array.cpp. So to use it in StatsMath, we need to include Array.hpp in StatsMath.hpp. Then, in the StatsMath methods, call `arr.getData()` to get the 2D vector and compute stats.

But the user hasn't mentioned modifying the Array class. However, in the previous answer, the Array class didn't have getters. So, perhaps the user expects that the StatsMath class can work with the existing Array class. But without getters, the StatsMath functions can't access the data. So, either:

1. Modify the Array class to have getters for rows, cols, and data.
2. Make StatsMath a friend of Array.
3. Or, if the user's code already has public access, but in the previous code, data was private.

In the previous code, data was private. So without getters, StatsMath can't access it. Therefore, to proceed, the Array class needs to be modified to include getter methods.

But since the user is following step by step, maybe they expect that the StatsMath class is part of the same project, so modifying the Array class is acceptable.

So, first, update the Array.hpp to include getters:

Add public methods in Array.hpp:

```
size_t getRows() const { return rows; }
```

```
size_t getCols() const { return cols; }
```

```
const std::vector<std::vector<double>>& getData() const { return data; }
```

This way, StatsMath can access the data via these getters.

Once that's done, the StatsMath class can be implemented.

Now, for each statistical function:

- Mean: sum all elements / total elements.
- Median: sort elements and find middle value.
- Sum: sum all elements.
- Product: product of all elements.
- Min/Max: find min and max values.
- Argmin/Argmax: find indices of min and max.
- Variance, Standard Deviation: compute using mean.
- Cumulative sum: return a 1D vector of cumulative sums.
- Percentile: sort and find value at a certain percentile.

Implementing these for a 2D array would require flattening the data, unless axis is considered. For simplicity, let's assume the functions work on the entire flattened array unless specified otherwise.

Alternatively, add an axis parameter, but that complicates things. Let's start with the entire array.

So, the StatsMath class will have static methods that take an Array and return a double (or a vector for cumsum).

Let's outline the methods:

- static double mean(const Array& arr);
- static double median(const Array& arr);
- static double sum(const Array& arr);
- static double prod(const Array& arr);
- static double min(const Array& arr);
- static double max(const Array& arr);
- static size_t argmin(const Array& arr);
- static size_t argmax(const Array& arr);
- static double variance(const Array& arr);
- static double stddev(const Array& arr);

- static std::vector<double> cumsum(const Array& arr);
- static double percentile(const Array& arr, double q);

Now, implement these in StatsMath.cpp.

For example, the mean function would flatten the array's data, sum all elements, and divide by the total number.

But to flatten, use the existing flatten method from the Array class. Wait, in the previous Array class, there's a flatten() method that returns a 1D vector. So, in StatsMath functions, we can call arr.flatten() to get the 1D vector and compute stats on that.

Wait, in the Array class, the flatten() method is public. So in StatsMath::mean, we can do:

```
std::vector<double> flat = arr.flatten();
if (flat.empty()) return 0.0;
double total = std::accumulate(flat.begin(), flat.end(), 0.0);
return total / flat.size();
```

Similarly for other functions.

So, the StatsMath methods can use the existing public methods of the Array class, like flatten(), which is already public.

Wait, in the previous Array.hpp, the flatten() method is public:

public:

...

```
std::vector<double> flatten() const;
```

So, the StatsMath functions can call arr.flatten() to get the 1D data.

Therefore, the StatsMath class doesn't need to access the private data of Array, as long as flatten() is available.

This simplifies things. So, no need to modify the Array class; just use the existing public methods.

Therefore, the StatsMath class can be implemented with static methods that take an Array, call flatten(), and compute stats on the 1D vector.

However, some functions like argmin and argmax need the original indices in the 2D array. For example, in NumPy, argmin returns the index in the flattened array by default, unless an axis is specified. So, if we follow that, then argmin would return the index in the flattened array.

But in the previous Array class, the flatten() method returns a 1D vector in row-major order. So, the StatsMath::argmin would return the index in this flattened vector.

But if the user wants the 2D indices, that's more complex. But for simplicity, let's return the flattened index, as per NumPy's default behavior.

Now, implement each function:

- Mean: sum of flattened / size.

- Median: sort flattened, return middle element.
- Sum: sum of all elements.
- Product: multiply all elements.
- Min: minimum element.
- Max: maximum element.
- Argmin: index of minimum in flattened.
- Argmax: index of maximum in flattened.
- Variance: $\text{sum}((x - \text{mean})^2) / (n - 1)$ for sample variance.
- Stddev: $\text{sqrt}(\text{variance})$.
- Cumsum: cumulative sum of flattened.
- Percentile: sort and interpolate.

Now, coding these.

Potential issues:

- Handling empty arrays: need to throw exceptions or return 0?
- For product, if the array is large, product can overflow. But using doubles might mitigate that.
- For percentile, interpolation method (e.g., linear, lower, etc.). Use NumPy's default method, which is linear.

But to keep it simple, implement the nearest rank method.

Wait, NumPy's default is linear interpolation. Let's implement that.

But implementing percentile correctly requires sorting the data, then finding the position via $(q/100)*(n-1)$, and interpolating between the surrounding values.

But for the sake of time and complexity, perhaps implement a simplified version.

Now, coding.

First, StatsMath.hpp:

```
#ifndef STATSMATH_HPP
#define STATSMATH_HPP
#include "Array.hpp"
#include <vector>
class StatsMath {
public:
static double mean(const Array& arr);
static double median(const Array& arr);
```

```

static double sum(const Array& arr);
static double prod(const Array& arr);
static double min(const Array& arr);
static double max(const Array& arr);
static size_t argmin(const Array& arr);
static size_t argmax(const Array& arr);
static double variance(const Array& arr, bool population = false);
static double stddev(const Array& arr, bool population = false);
static std::vector<double> cumsum(const Array& arr);
static double percentile(const Array& arr, double q);
};
#endif

```

Then, StatsMath.cpp:

```

#include "StatsMath.hpp"
#include <vector>
#include <algorithm>
#include <numeric>
#include <cmath>
using namespace std;

double StatsMath::mean(const Array& arr) {
    vector<double> flat = arr.flatten();
    if (flat.empty()) throw invalid_argument("Array is empty.");
    return accumulate(flat.begin(), flat.end(), 0.0) / flat.size();
}

double StatsMath::median(const Array& arr) {
    vector<double> flat = arr.flatten();
    if (flat.empty()) throw invalid_argument("Array is empty.");
    size_t n = flat.size();
    sort(flat.begin(), flat.end());
    if (n % 2 == 0) {
        return (flat[n/2 - 1] + flat[n/2]) / 2.0;
    } else {

```

```

return flat[n/2];
}
}

double StatsMath::sum(const Array& arr) {
vector<double> flat = arr.flatten();
return accumulate(flat.begin(), flat.end(), 0.0);
}

double StatsMath::prod(const Array& arr) {
vector<double> flat = arr.flatten();
if (flat.empty()) return 0.0;
double product = 1.0;
for (double val : flat) {
product *= val;
}
return product;
}

double StatsMath::min(const Array& arr) {
vector<double> flat = arr.flatten();
if (flat.empty()) throw invalid_argument("Array is empty.");
return *min_element(flat.begin(), flat.end());
}

double StatsMath::max(const Array& arr) {
vector<double> flat = arr.flatten();
if (flat.empty()) throw invalid_argument("Array is empty.");
return *max_element(flat.begin(), flat.end());
}

size_t StatsMath::argmin(const Array& arr) {
vector<double> flat = arr.flatten();
if (flat.empty()) throw invalid_argument("Array is empty.");
return distance(flat.begin(), min_element(flat.begin(), flat.end()));
}

size_t StatsMath::argmax(const Array& arr) {

```



```

vector<double> flat = arr.flatten();
if (flat.empty()) throw invalid_argument("Array is empty.");
return distance(flat.begin(), max_element(flat.begin(), flat.end()));
}

double StatsMath::variance(const Array& arr, bool population) {
vector<double> flat = arr.flatten();
if (flat.empty()) throw invalid_argument("Array is empty.");
double m = mean(arr);
double sum_sq = 0.0;
for (double val : flat) {
sum_sq += (val - m) * (val - m);
}
size_t n = flat.size();
return population ? sum_sq / n : sum_sq / (n - 1);
}

double StatsMath::stddev(const Array& arr, bool population) {
return sqrt(variance(arr, population));
}

vector<double> StatsMath::cumsum(const Array& arr) {
vector<double> flat = arr.flatten();
vector<double> result(flat.size());
partial_sum(flat.begin(), flat.end(), result.begin());
return result;
}

double StatsMath::percentile(const Array& arr, double q) {
vector<double> flat = arr.flatten();
if (flat.empty()) throw invalid_argument("Array is empty.");
if (q < 0 || q > 100) throw invalid_argument("Percentile must be between 0 and 100.");
sort(flat.begin(), flat.end());
double index = (q / 100.0) * (flat.size() - 1);
size_t lower = floor(index);
size_t upper = ceil(index);

```

```

if (lower == upper) {
return flat[lower];
}

double weight = index - lower;

return flat[lower] * (1 - weight) + flat[upper] * weight;
}

```

Then, the main.cpp would demonstrate these functions.

But wait, the user's Array class from previous steps uses a 2D vector. So when we call flatten(), it's a 1D vector of all elements. Therefore, the statistical functions are computed over all elements, regardless of the original dimensions.

Now, the user might want to handle axis, but that's more complex. For now, proceed with the entire array.

Finally, compile the code, ensuring that StatsMath.hpp includes Array.hpp, and all cpp files are compiled together.

Potential issues:

- The percentile function uses linear interpolation, which might not match NumPy exactly but is a reasonable approximation.
- The variance function uses Bessel's correction (divided by n-1) by default, which is sample variance. The population parameter can toggle between population and sample.
- The product function returns 0 if the array is empty, but maybe should throw an error. Need to handle edge cases.

Testing is important, but the code provides a foundational structure.

(Due to technical issues, the search service is temporarily unavailable.)

Je vais te montrer comment implémenter en C++ une classe pour les **statistiques et opérations mathématiques** inspirée de NumPy, en la séparant en fichiers .hpp et .cpp. Nous utiliserons la classe Array précédente comme base.

StatsMath.hpp (Déclaration des fonctions statistiques)

cpp
Copy

```

#ifndef STATSMATH_HPP
#define STATSMATH_HPP

#include "Array.hpp" // Inclure la classe Array précédente

class StatsMath {
public:
    // Fonctions statistiques de base
    static double mean(const Array& arr);

```

```

static double median(const Array& arr);
static double sum(const Array& arr);
static double prod(const Array& arr);
static double min(const Array& arr);
static double max(const Array& arr);

// Fonctions avancées
static double variance(const Array& arr, bool population = false);
static double stddev(const Array& arr, bool population = false);
static std::vector<double> cumsum(const Array& arr);
static double percentile(const Array& arr, double q);

// Indices et positions
static size_t argmin(const Array& arr);
static size_t argmax(const Array& arr);
};

#endif

```

StatsMath.cpp (Implémentation des fonctions)

cpp

Copy

```

#include "StatsMath.hpp"
#include <algorithm>
#include <numeric>
#include <cmath>
#include <stdexcept>

// Moyenne des éléments du tableau
double StatsMath::mean(const Array& arr) {
    auto flat = arr.flatten();
    if (flat.empty()) throw std::invalid_argument("Array is empty.");
    return std::accumulate(flat.begin(), flat.end(), 0.0) / flat.size();
}

// Médiane des éléments
double StatsMath::median(const Array& arr) {
    auto flat = arr.flatten();
    if (flat.empty()) throw std::invalid_argument("Array is empty.");
    std::sort(flat.begin(), flat.end());
    size_t n = flat.size();
    return (n % 2 == 0) ? (flat[n/2 - 1] + flat[n/2]) / 2.0 : flat[n/2];
}

// Somme des éléments
double StatsMath::sum(const Array& arr) {
    auto flat = arr.flatten();
    return std::accumulate(flat.begin(), flat.end(), 0.0);
}

// Produit des éléments
double StatsMath::prod(const Array& arr) {
    auto flat = arr.flatten();
    if (flat.empty()) return 0.0;
    return std::accumulate(flat.begin(), flat.end(), 1.0,
std::multiplies<double>());
}

// Valeur minimale

```

```

double StatsMath::min(const Array& arr) {
    auto flat = arr.flatten();
    if (flat.empty()) throw std::invalid_argument("Array is empty.");
    return *std::min_element(flat.begin(), flat.end());
}

// Valeur maximale
double StatsMath::max(const Array& arr) {
    auto flat = arr.flatten();
    if (flat.empty()) throw std::invalid_argument("Array is empty.");
    return *std::max_element(flat.begin(), flat.end());
}

// Indice de la valeur minimale (aplati)
size_t StatsMath::argmin(const Array& arr) {
    auto flat = arr.flatten();
    if (flat.empty()) throw std::invalid_argument("Array is empty.");
    return std::distance(flat.begin(), std::min_element(flat.begin(),
flat.end()));
}

// Indice de la valeur maximale (aplati)
size_t StatsMath::argmax(const Array& arr) {
    auto flat = arr.flatten();
    if (flat.empty()) throw std::invalid_argument("Array is empty.");
    return std::distance(flat.begin(), std::max_element(flat.begin(),
flat.end()));
}

// Variance (population = false pour variance échantillon)
double StatsMath::variance(const Array& arr, bool population) {
    double m = mean(arr);
    auto flat = arr.flatten();
    double sum_sq = std::accumulate(flat.begin(), flat.end(), 0.0,
    [m](double acc, double x) { return acc + (x - m) * (x - m); });
    return sum_sq / (flat.size() - (population ? 0 : 1));
}

// Écart-type
double StatsMath::stddev(const Array& arr, bool population) {
    return std::sqrt(variance(arr, population));
}

// Somme cumulative
std::vector<double> StatsMath::cumsum(const Array& arr) {
    auto flat = arr.flatten();
    std::vector<double> result(flat.size());
    std::partial_sum(flat.begin(), flat.end(), result.begin());
    return result;
}

// Percentile (approche linéaire comme dans NumPy)
double StatsMath::percentile(const Array& arr, double q) {
    auto flat = arr.flatten();
    if (flat.empty()) throw std::invalid_argument("Array is empty.");
    std::sort(flat.begin(), flat.end());
    double index = (q / 100.0) * (flat.size() - 1);
    size_t lower = static_cast<size_t>(std::floor(index));
    size_t upper = static_cast<size_t>(std::ceil(index));
    double weight = index - lower;
    return flat[lower] * (1 - weight) + flat[upper] * weight;
}

```

main.cpp (Exemple d'utilisation)

cpp

Copy

```
#include "Array.hpp"
#include "StatsMath.hpp"
#include <iostream>

int main() {
    // Créer un tableau 2x3
    Array arr({{1, 2, 3}, {4, 5, 6}});

    // Calculer des statistiques
    std::cout << "Mean: " << StatsMath::mean(arr) << std::endl;      // 3.5
    std::cout << "Median: " << StatsMath::median(arr) << std::endl;    // 3.5
    std::cout << "Sum: " << StatsMath::sum(arr) << std::endl;          // 21
    std::cout << "Max: " << StatsMath::max(arr) << std::endl;          // 6
    std::cout << "Argmin: " << StatsMath::argmin(arr) << std::endl;    // 0
    (indice de 1)

    // Variance et écart-type
    std::cout << "Variance: " << StatsMath::variance(arr) << std::endl; //
3.5 (échantillon)
    std::cout << "Std Dev: " << StatsMath::stddev(arr) << std::endl;    //
~1.8708

    // Percentile
    std::cout << "25th percentile: " << StatsMath::percentile(arr, 25) <<
std::endl; // ~2.25

    return 0;
}
```

Compilation et exécution

Compile avec :

bash

Copy

```
g++ -std=c++11 Array.cpp StatsMath.cpp main.cpp -o stats_demo
```

Exécute avec :

bash

Copy

```
./stats_demo
```

Explications

1. Classe **StatsMath** :

- **Statiques** : Toutes les méthodes sont statiques, car elles opèrent sur des objets Array.
- **Flattening** : Les fonctions utilisent `arr.flatten()` pour traiter le tableau 2D comme un vecteur 1D.
- **Gestion d'erreurs** : Lance des exceptions si le tableau est vide.

2. Fonctionnalités implémentées :

- **Moyenne, médiane, somme, produit** : Opérations de base.
- **Variance/écart-type** : Avec option pour population ou échantillon.
- **Percentile** : Implémentation similaire à NumPy avec interpolation linéaire.
- **Argmin/argmax** : Retourne l'indice dans le tableau aplati.

3. Améliorations possibles :

- Ajouter la gestion des axes (ex: `axis=0` ou `axis=1` comme dans NumPy).
- Implémenter des fonctions comme `cumprod`, `nanmean`, etc.
- Optimiser pour les grands tableaux (utilisation de la mémoire).

Si tu veux ajouter d'autres fonctions ou modifier le comportement, dis-le-moi ! 😊

donc premierement faire 4 Algèbre Linéaire avec cpp et faire comme objet separé dans deux fichier .hpp an .cpp